

Selection Search for Mean and Temperature of Multi-Branch Combinatorial Games

Kuo-Yuan Kao

Department of Information Management
National Penghu University, Penghu, Taiwan
stone@npu.edu.tw

Abstract

This paper shows a new algorithm to calculate the mean and temperature of multi-branch combinatorial games. The algorithm gradually, one node at a time, expands the offspring of a game. After each step of expansion, the lower and upper bounds of the mean and temperature of the game is re-calculated. As the expanding process continues, the range between the lower and upper bounds is narrowed. The key feature of the algorithm is its ability to generate a path whose outcome is most likely to reduce the distance between the lower and upper bounds.

1 Introduction

For each combinatorial game, there are two important values, *mean* and *temperature*. Roughly speaking, mean is a measure of the average outcome and temperature is a measure of the move size of a game. The existence of mean values of games was first raised and proved by Milnor[1] and Hanner[2]. A constructive algorithm, named thermograph, for mean and temperature was due to Berlekamp[3] and Conway[4]. An approach for calculating mean and temperature with partial information of a single branch game was proposed by Kao[5].

In this paper, the mean and temperature of a game G is denoted as $m(G)$ and $t(G)$.

Let G be a game and t be a number, define

$$L(G, t) = \max_{\{x \in G^L\}} \begin{cases} m(G) & \dots\dots\dots(1) \\ R(x, t) - t \end{cases}$$

$$R(G, t) = \min_{\{y \in G^R\}} \begin{cases} m(G) & \dots\dots\dots(2) \\ L(y, t) + t \end{cases}$$

$L(G, t)$ is called the *left wall* and $R(G, t)$ the *right wall* of G . Finding the mean and temperature of a game G is indeed a task of solving the min-max equation below.

$$\max_{\{x \in G^L\}} R(x, t) - t = \min_{\{y \in G^R\}} L(y, t) + t \dots\dots\dots(3)$$

There might be more than one solution of t for the above equation. The minimum solution of t is $t(G)$. When $t=t(G)$, the value of the min-max equation equals $m(G)$.

2 Thermograph

This section reviews the *thermograph* [3][4] approach for calculating the mean and temperature of a game.

A function $f(t)$ is called *simple max* if it can be written as:

$$f(t) = \max \{c_1, \min \{c_2 - t, \dots \max \{c_{2k-1}, \min \{c_{2k} - t, \dots\}\}\} \dots\} \dots(4)$$

where $c_{2k} > c_{2k+2}$, $c_{2i} > c_{2k+1} > c_{2k-1}$.

Similarly, $g(t)$ is called *simple min* if it can be written as:

$$g(t) = \min \{c_1, \max \{c_2 + t, \dots \min \{c_{2k-1}, \max \{c_{2k} + t, \dots\}\}\} \dots\} \dots(5)$$

where $c_{2k-1} > c_{2k+1} > c_{2i}$, $c_{2k+2} > c_{2k}$.

Each simple max(min) function can be represented as a sequence $[c_1, \dots, c_n]$ of constants. The graph of a simple max(min) function is a folded line. Figure 1(a) and 1(b) show the graph of simple max and min functions.

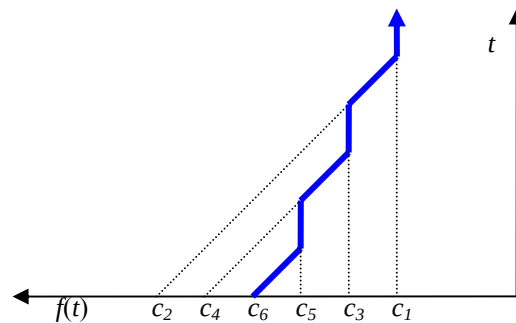


Figure 1(a): graph of simple max function

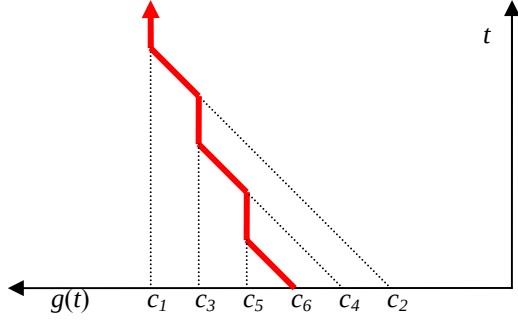
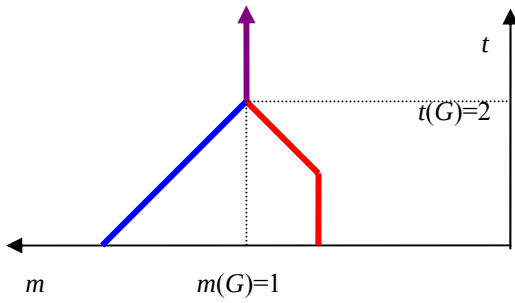


Figure 1(b): graph of simple min function

It should be clear that the $\max(\min)$ of two simple $\max(\min)$ functions is again a simple $\max(\min)$ function. If $f(t)$ is a simple $\max$ function and c is a number, then $\min\{c, f(t)+t\}$ is a simple $\min$ function. If $g(t)$ is a simple $\min$ function and c is a number, then $\max\{c, g(t)-t\}$ is a simple $\max$ function. Thus, the left wall of a game is a simple $\max$ function and the right wall is a simple $\min$ function. Figure 2 shows the thermograph of $G = \{3|\{0|-2\}\}$.


 Figure 2: thermograph of $G = \{3|\{0|-2\}\}$

The general procedure to calculate the walls, LW and RW, of a game G is as follows.

1. Calculate the walls of all G 's children.
2. Find the $\max$ of the RWs of G 's left children. Store the result as $R(t)$.
3. Find the $\min$ of the LWs of G 's right children. Store the result as $L(t)$.
4. Calculate $m(G)$
[solve the equation $R(t)-t=L(t)+t$.]
5. $LW = \max\{m(G), R(t)-t\}$
6. $RW = \min\{m(G), L(t)+t\}$

The above procedure is recursive. To calculate the walls of G , one needs to calculate the walls of all G 's children first (step 1). Eventually, the walls of all the offspring of G must be calculated in order to calculate the walls of G .

3 Upper and Lower Walls

In order to calculate the walls of a game, the thermograph approach requires visiting all the offspring of the game. In many applications, the number of a game's nodes could be a quite huge number, which makes the thermograph approach infeasible. However, partial information of a game's nodes could be used to estimate the lower and upper bounds of the game's walls (or mean and temperature). For example, consider the game $H = \{\{x|20\} | 4\}$. Although $H^{LL} = x$ has unknown value, as long as $x \geq 20$, it can be shown $12 \leq m(H) \leq 20$ and $8 \leq t(H) \leq 16$. Sometimes, partial information of a game's nodes could be sufficient to determine its mean and temperature. For example, consider the game $G = \{\{\{y|20\}|4\} | 0\}$. Although $G^{LLL} = y$ has unknown value, as long as $y \geq 20$, it can be shown $m(G)=4$ and $t(G)=4$.

For a hot combinatorial game G , the temperature is at least 0. It can be deduced

$$m(G^L) \geq R(G^L, 0) = L(G, 0) \geq R(G, 0) \dots \dots \dots (6)$$

$$m(G^R) \leq L(G^R, 0) = R(G, 0) \leq L(G, 0) \dots \dots \dots (7)$$

(6) and (7) can be used to setup the lower or upper bound of a missing left or right child of a game. The assumptions $x \geq 20$ and $y \geq 20$ in the previous examples are reasonable provided the positions are hot games.

Let $m^U(G)$ and $m^L(G)$ denotes the upper and lower bounds of $m(G)$, $t^U(G)$ and $t^L(G)$ denotes the upper and lower bounds of $t(G)$. Define

$$L^U(G, t) = \max_{\{x \in G^L\}} \left\{ \begin{array}{l} m^U(G) \\ R^U(x, t) - t \end{array} \right\} \dots \dots \dots (8)$$

$$L^L(G, t) = \max_{\{x \in G^L\}} \left\{ \begin{array}{l} m^L(G) \\ R^L(x, t) - t \end{array} \right\} \dots \dots \dots (9)$$

$$R^U(G, t) = \min_{\{y \in G^R\}} \left\{ \begin{array}{l} m^U(G) \\ L^U(y, t) + t \end{array} \right\} \dots \dots \dots (10)$$

$$R^L(G, t) = \min_{\{y \in G^R\}} \left\{ \begin{array}{l} m^L(G) \\ L^L(y, t) + t \end{array} \right\} \dots \dots \dots (11)$$

These are the upper and lower bounds of the left and right walls of G . If one knows the upper and lower bounds of the means of all the offspring of G , then $m^U(G)$, can be derived by solving t of the equation

$$\max_{\{x \in G^L\}} R^U(x, t) - t = \min_{\{y \in G^R\}} L^U(y, t) + t \dots\dots (12)$$

$m^L(G)$, can be derived by solving t of the equation

$$\max_{\{x \in G^L\}} R^L(x, t) - t = \min_{\{y \in G^R\}} L^L(y, t) + t \dots\dots (13)$$

$t^U(G)$, can be derived by solving t of the equation

$$\max_{\{x \in G^L\}} R^U(x, t) - t = \min_{\{y \in G^R\}} L^L(y, t) + t \dots\dots (14)$$

$t^L(G)$, can be derived by solving t of the equation

$$\max_{\{x \in G^L\}} R^L(x, t) - t = \min_{\{y \in G^R\}} L^U(y, t) + t \dots\dots (15)$$

Figure 3 shows the relation between upper and lower walls and bounds of mean and temperature.

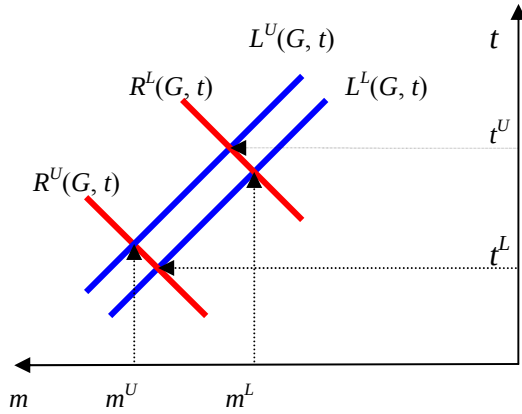


Figure 3: relation between upper and lower walls and bounds of mean and temperature

4 MT-Search

This section presents a new algorithm, *MT-search*, to calculate the mean and temperature of games. The algorithm gradually, one node at a time, expands the offspring of a game. After each step of expanding, the lower and upper walls of all the nodes on the path from the new node to the root are re-calculated. As the expanding process continues, the distance between the lower and upper walls is narrowed. The algorithm terminates when either the distance between the upper and lower walls becomes 0 or the maximum allowed number of visited nodes is reached. The choosing of an offspring node to expand is determined by some selection rules. These rules are introduced in later sections.

The MT-search algorithm is implemented in a game-independent engine named *MT-engine*.

Application games can communicate with the engine through MT-engine's interface methods.

```
class MT_engine
{
    constructor MT_engine();
    destructor ~MT_engine();
    void explore(char path[]);
    void add_node(char path[], float value);
    float mean_UB();
    float mean_LB();
    float temp_UB();
    float temp_LB();
};
```

An application game can start an instance game by invoking the constructor of MT-engine. At the beginning, there is no visited node of the instance game. At each run, the application game calls the `explore()` method of MT-engine to get a path to be explored, and calls the `add_node()` method to add a node to the instance game of MT-engine. The MT-engine will update the upper and lower bounds of the mean and temperature each time after a new node been added to the instance game. At any time, the methods `mean_UB`, `mean_LB`, `temp_UB`, and `temp_LB` returns the upper and lower bounds of the mean and temperature of the instance game.

The application game must provide a method `outcome(char path[])`, which returns the outcome of the specified path. The path is a string in the format $D_1n_1D_2n_2\dots D_kn_k\dots D_mn_m$, where $D_k \in \{L, R\}$ is the direction of the child and n_k is an integer indicating the branching order of the child. For example $L3$ indicates the 3rd left child of the root, $L3R2$ indicates the 2nd right child of the 3rd left child of the root. If the specified path does not exist in the application then `outcome()` returns a special value `NOT_EXISTS`. Otherwise, when the given path is not a terminal node, the `outcome()` method automatically extends the path until it reaches a terminal node. The extended path must be in alternating directions and always selects the first child. For example, if the path $L1L2R3$ is not a terminal node then the path will be extended to $L1L2R3L1$, $L1L2R3L1R1$, $L1L2R3L1R1L1$... until it reaches a terminal node.

Below is an example of feeding MT-engine with 8 paths of a game with 10 or more nodes. The paths are generated by the engine. The application game inputs the values of the paths. With partial information of the game, the engine still can estimate the ranges of the mean and temperature of the game. After input 8 paths, the engine concludes the mean and temperature of the game. Figure 4 shows known paths of the game.

- | | | |
|----------------|---------------------------|--------------------------|
| (1) L1R1L1=8 | $m = [-\text{INF}, 8.00]$ | $t = [0.00, \text{INF}]$ |
| (2) R1L1=-2 | $m = [-2.00, 8.00]$ | $t = [5.00, \text{INF}]$ |
| (3) R2L1R1=2 | $m = [-2.00, 8.00]$ | $t = [5.00, \text{INF}]$ |
| (4) R1R1L1=-10 | $m = [-1.00, 8.00]$ | $t = [7.00, \text{INF}]$ |
| (5) L2R1=-2 | $m = [-1.00, 8.00]$ | $t = [7.00, \text{INF}]$ |
| (6) L1L1=16 | $m = [-1.00, 3.00]$ | $t = [7.00, 11.00]$ |
| (7) R1R1R1=-16 | $m = [0.25, 2.25]$ | $t = [7.75, 10.00]$ |
| (8) L1R1R1=2 | $m = [1.50, 1.50]$ | $t = [9.00, 9.00]$ |

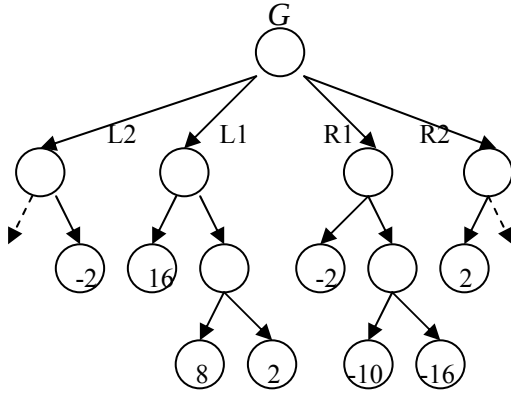


Figure 4: a game with incomplete information

5 Uncertainty and Stability

MT-engine has no specific domain knowledge about the application games. The key feature of MT-engine is its ability to generate a path whose outcome is most likely to reduce the distance between the lower and upper walls.

The first task of MT-engine is to decide the direction, left or right, to explore. The decision is based on the uncertainties of the walls and the stability of the current node.

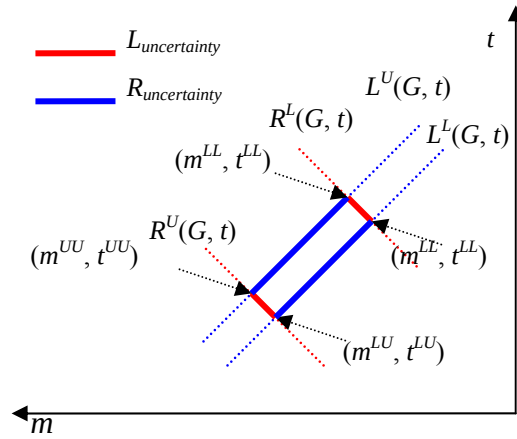
We define left and right *uncertainty* of G as:

$$L_{\text{uncertainty}}(G) = \sqrt{(m^{UL} - m^{LL})^2 + (t^{UL} - t^{LL})^2} + \sqrt{(m^{UU} - m^{LU})^2 + (t^{UU} - t^{LU})^2} \dots (16)$$

$$R_{\text{uncertainty}}(G) = \sqrt{(m^{UL} - m^{UU})^2 + (t^{UL} - t^{UU})^2} + \sqrt{(m^{LL} - m^{LU})^2 + (t^{LL} - t^{LU})^2} \dots (17)$$

where (m^{UU}, t^{UU}) , (m^{LL}, t^{LL}) , (m^{UL}, t^{UL}) and (m^{LU}, t^{LU}) are the solutions of equations (12) to (15).

$L_{\text{uncertainty}}(R_{\text{uncertainty}})$ measures the distance between the lower and upper left(right) walls. The higher the uncertainty value is, the more likely a path in that direction will return useful information. Thus, MT-engine tends to select the direction with greater uncertainty value. Figure 5 shows the uncertainty values in a thermograph. In figure 5, $L_{\text{uncertainty}}$ is less than $R_{\text{uncertainty}}$. The uncertainty value will reduce to 0 once the upper wall matches the lower wall.


 Figure 5: $L_{\text{uncertainty}}$ and $R_{\text{uncertainty}}$

We say a game G is *stable* under t provided $t(G) < t$, and *unstable* provided $t(G) \geq t$. In our implementation, the value t is the minimum of the temperatures of G 's ancestors. Consider game G as a right child of its parent.

- When $t(G) < t$, $L(G, t) = m(G)$ the value of G^R will impact the value of $m(G)$, hence still need to visit the nodes in G^R .
- When $t(G) \geq t$, $L(G, t) = \max R(G^L, t) - t$, the value of G^R will not impact the left wall of G , and thus the right wall of G 's parent. When this happens, there is no need to visit the nodes in G^R .

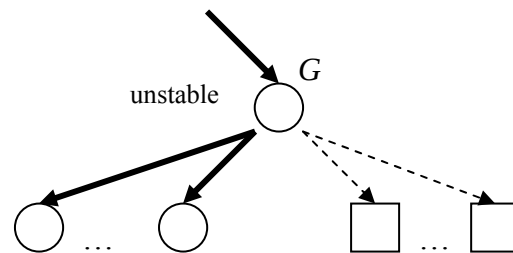


Figure 6: exploring an unstable node

Similarly, if G is an unstable left child of its parent, there is no need to visit the nodes in G^L .

In general, one may not be able to determine whether a node is stable or not during the search. We define the *stability* of G under t is defined as:

$$p_{stable}(G, t) = \begin{cases} 0, & \text{if } t^L(G) \geq t \\ 1, & \text{if } t^U(G) < t \\ (t - t^L(G)) / (t^U(G) - t^L(G)), & \text{otherwise} \end{cases} \dots\dots\dots(18)$$

Stability and uncertainty can be combined together to determine the direction of exploration.

If G is a right child then explore its left children when

$$L_{uncertainty}(G) \geq R_{uncertainty}(G) \times p_{stable}(G, t) \dots\dots\dots(19)$$

Otherwise, explore right children.

If G is a left child then explore its right children when

$$R_{uncertainty}(G) \geq L_{uncertainty}(G) \times p_{stable}(G, t) \dots\dots\dots(20)$$

Otherwise, explore its left children.

Note that the values of uncertainty and stability depend on the bounds of $m(G)$ and $t(G)$. In real implementation, the bounds of $m(G)$ and $t(G)$ may not be accurate during the search. Especially, when either uncertainty or stability becomes 0, the search algorithm will block one direction to explore. To avoid this problem, one can introduce some noise to both sides of (19) and (20).

6 Minimum Proving Nodes

MT-search can produce an estimation of the mean and temperature of a game even if there are only a few visited nodes. But how good is the quality? In this section, we discuss the number of nodes required to produce a meaningful estimation.

Let G be a game with branching factor m (each player has m options at any none-terminal node) and depth n . The total nodes $T(m, n)$ of G is

$$T(m, n) = (2m)^n, \quad m > 0, \quad n > 0 \dots\dots\dots(19)$$

Define $f(m, n)$ the least number of nodes required to prove the optimal left outcome, $L(G, 0)$. Then

$$f(m, 0) = 1,$$

$$f(m, 1) = m,$$

$$f(m, n) = f(m, n-1) + (m-1) \times f(m, n-2), \quad n > 1 \dots\dots\dots(20)$$

Define $s(m, n)$ the least number of nodes required to prove $m(G)$, $u(m, n)$ the least number of nodes required to prove G unstable, $p(m, n)$ the least number of nodes required to prove primary node G . Then

$$s(m, 0) = 1$$

$$s(m, 1) = 2m$$

$$s(m, n) = 2 \times [p(m, n-1) + (m-1) \times f(m, n-2)], \quad n > 1 \dots\dots\dots(21)$$

$$u(m, 0) = 1$$

$$u(m, 1) = m + 1$$

$$u(m, n) = f(m, n) + p(m, n-1), \quad n > 1 \dots\dots\dots(22)$$

For each primary node G , one either proves the value of $m(G)$ (when G is stable) or proves G unstable. Assume the odds for a primary node been stable and unstable is even, then

$$p(m, n) = [s(m, n) + u(m, n)] / 2, \quad n > 0 \dots\dots\dots(23)$$

Table 1 shows the values of the ratios $c(m, n) = s(m, n) / \sqrt{T(m, n)}$ for a selected ranges of m and n . When m is between 2 and 5, and n is less than 10, $s(m, n)$ is about the same order of $\sqrt{T(m, n)}$. In general, $c(m, n)$ decreases as n increases. This result indicates that, to produce an accurate output of mean and temperature, any algorithm needs to visit at least some multiples of $\sqrt{T(m, n)}$, when m, n are in the ranges in table 1.

Table 1: $c(m, n) = s(m, n) / \sqrt{T(m, n)}$

n\m	2	3	4	5
1	2.00	2.45	2.83	3.16
2	2.25	2.33	2.38	2.40
3	2.44	2.45	2.50	2.56
4	2.39	2.19	2.09	2.02
5	2.26	1.97	1.84	1.79
6	2.07	1.69	1.52	1.43
7	1.86	1.44	1.28	1.19
8	1.64	1.21	1.05	0.96
9	1.43	1.01	0.86	0.79
10	1.23	0.84	0.71	0.64

7 Exploration and Exploitation

Once one determined the direction to explore, the next step of MT-engine is to select a child in the direction to explore. The general principle is selecting moves in the tree such that good moves are searched more often than moves that appear to be bad. One way of doing this is defining the weighted frequency as below, and selecting the child with minimum weighted frequency to explore.

```
if(b_primary(child)) child.freq = child.visits;
else                  child.freq=child.visits*K;
```

In our implementation, K depends on the stability of the *primary* child, the *branching factor* m and depth n of the *primary* child. When the primary child is stable,

$$K_s = s(m, n) / f(m, n-1) \dots \dots \dots (24)$$

When the primary child is unstable,

$$K_u = u(m, n) / f(m, n-1) \dots \dots \dots (25)$$

The general formula is

$$K = (K_s \times p_{stable}) + (K_u \times (1 - p_{stable})) \dots \dots \dots (26)$$

Table 2(a) and 2(b) show the values of K_s and K_u for a selected range of m and n . Note that (1) for a given m and n , K_s is about 50% greater than K_u , and (2) both K_s and K_u increase as the depth n increases.

In practice, the branching factor and depth of a game G may not be available. These parameters may be estimated by statics of the visited nodes of G . The default value can be $K_s=9$ and $K_u=6$.

Table 2(a): $K_s(m, n)$

n\m	2	3	4	5
1	4.00	6.00	8.00	10.00
2	4.50	4.67	4.75	4.80
3	6.50	7.20	8.07	9.00
4	7.90	7.36	7.18	7.10
5	9.63	8.67	8.68	8.98
6	11.19	9.07	8.46	8.24
7	12.83	9.75	9.06	8.99
8	14.44	10.12	9.05	8.71
9	16.06	10.52	9.29	9.00
10	17.68	10.78	9.32	8.90

Table 2(b): $K_u(m, n)$

n\m	2	3	4	5
1	3.00	4.00	5.00	6.00
2	3.25	3.33	3.38	3.40
3	4.67	5.00	5.43	5.89
4	5.50	5.18	5.08	5.03
5	6.56	5.90	5.84	5.95
6	7.54	6.21	5.81	5.66
7	8.55	6.60	6.12	6.02
8	9.54	6.84	6.15	5.92
9	10.55	7.08	6.28	6.06
10	11.54	7.25	6.31	6.02

8 Heap Game

This section introduces a heap game, named *heap-go*, for testing the performance of the MT-search algorithm. Heap-go is played on a number of heaps of counters. Each counter has a weight and is colored either blue or red.

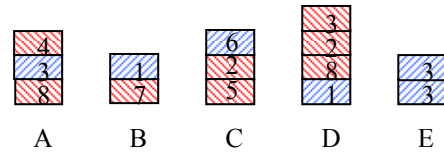


Figure 7: a setup of heap-go

Two players, L and R , move alternatively and their legal moves are different.

- When it is L 's turn to move, he can choose any one of the heaps and repeatedly removes the top counter until either he removed a red counter or the heap has become empty.
- When it is R 's turn to move, he can choose any one of the heaps and repeatedly removes the top counter until either he removed a blue counter or the heap has become empty.

The game is finished once all the counters in all the heaps were removed. The player who removed more total weights is the winner.

Figure 8 shows the game tree of heap C in figure 7. The numbers at the terminal nodes are the net scores of the paths from the root to these nodes. L 's score are counted positive and R 's negative. For example, consider the path LR. L gets 8 points for the first move (removed 2 counters); R gets 5 points for the second move (removed 1 counter); the net score is 3.

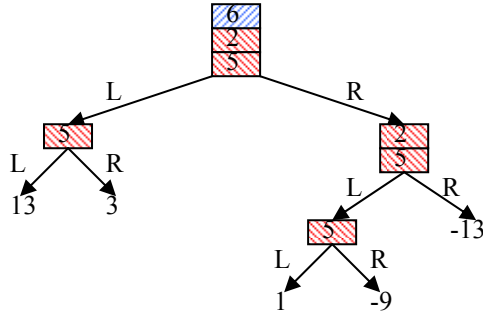


Figure 8: game tree of heap C

Heap-go resembles go endgame a lot. Each heap has a mean and a temperature. Usually, players try to find the heap with the maximum temperature to move. For 19x19 go endgames, the temperatures of endgame positions range roughly from 1 to 20. For heap-go, the temperatures of heaps also range from 1 to 20 provided the weights of counters range from 1 to 10.

The calculation of the mean and temperature of a single heap is not a difficult task, because there are only n distinct states in the game tree of a heap with n counters and there is only one option for each player at every none-terminal position of the game tree. An efficient algorithm is provided in [5] to calculate the mean and temperature of a heap.

Our goal here is not to solve or analyze heap-go, but to use it as a sample space for testing the performance of the MT-search. There are several reasons to use heap-go as a sample space.

1. The maximum branching factor of a sample game can be controlled. A sum of m heaps has branching factor m .
2. The maximum depth of a sample game can be controlled. A sum of heaps with total n counters has maximum depth n .
3. The range of the temperature of a sample game can be controlled. A sum of heaps with counter weights ranging from 1 to w has a temperature range from 1 to $2w$.

9 Experiments Result

Table 3 shows the classes of sums of heaps used as sample space in our experiments. The branching factor ranges from 2 to 5; the maximum depth ranges from 6 to 14; the temperatures range from 1 to 20. These ranges are chosen to resemble the branching factor, depth and temperature of endgame positions of 19x19 go. The last column is the average total nodes for each class.

Table 3: classes of sums of heaps

class (heap sizes)						T
A	3	3				198
	4	4				1,092
	5	5				6,215
	6	6				35,720
	7	7				205,590
B	2	2	2			1,158
	3	2	2			3,281
	3	3	2			10,166
	3	3	3			34,533
	4	3	3			103,745
C	2	2	1	1		3,936
	2	2	2	1		65,304
	3	3	2	1		167,055
D	2	1	1	1	1	13,440
	2	2	1	1	1	52,800
	2	2	2	1	1	230,880

For each class, 1,000 sample games are randomly generated. There are two types of sample games: the first type has random branches, while the second type has ordered branches. When the branches are ordered, they are ordered by the heap temperatures. For each sample game, the MT-engine visits a set of predefined numbers of nodes. The set of predefined numbers of nodes are in the form:

$$2\sqrt{m} \times T^d \dots \dots \dots (27)$$

where m is the number of heaps in the sample game, T is the total nodes of the sample game and d ranges from 0.5 to 0.75. The parameters are chosen to guarantee the number of visited nodes no less than the minimum proving nodes as discussed in section 6.

The output temperature of MT-search is compared with the exact temperature of each game, and the square errors of all sample games in the same class are summed. Finally, the mean square error and the standard error are calculated for each class.

Table 4 shows the standard error for each class, where sample games have random branches. The results are summarized as follows.

- When $d=0.5$, the number of visited nodes is about the order of the number of minimum proving nodes, the output of MT-search has a standard error of 0.93 in average.
- When the value of d increased by 0.05, the standard error is decreased by about 20 to 25%.

Table 4: standard error for each class (random branches)

Class		D					
		0.50	0.55	0.60	0.65	0.70	0.75
A	33	1.08	0.88	0.56	0.43	0.30	0.08
	44	0.95	0.83	0.50	0.47	0.25	0.23
	55	0.81	0.74	0.42	0.25	0.18	0.10
	66	0.76	0.63	0.46	0.34	0.18	0.18
	77	0.68	0.64	0.22	0.12	0.11	0.07
B	222	1.05	0.67	0.46	0.39	0.16	0.14
	322	0.93	0.81	0.53	0.30	0.20	0.18
	332	0.99	0.69	0.47	0.26	0.25	0.07
	333	0.77	0.66	0.28	0.23	0.20	0.19
	433	0.89	0.70	0.45	0.24	0.19	0.13
C	2211	1.06	0.84	0.68	0.38	0.21	0.21
	2221	0.90	0.62	0.54	0.35	0.29	0.17
	3321	1.18	0.75	0.44	0.34	0.24	0.19
D	21111	1.04	0.58	0.32	0.33	0.20	0.08
	22111	0.97	0.66	0.49	0.31	0.26	0.08
	22211	0.78	0.59	0.42	0.27	0.22	0.17
average		0.93	0.71	0.45	0.31	0.22	0.14

With sample space temperature ranging from 1 to 20, a standard error of less than 1.0 is a quite promising result.

Table 5: standard error for each class (ordered branches)

Class		D					
		0.50	0.55	0.60	0.65	0.70	0.75
A	33	0.92	0.55	0.32	0.15	0.05	0.01
	44	0.58	0.39	0.30	0.14	0.04	0.00
	55	0.45	0.26	0.15	0.09	0.02	0.00
	66	0.39	0.17	0.16	0.03	0.01	0.00
	77	0.24	0.12	0.03	0.01	0.00	0.00
B	222	0.58	0.47	0.20	0.09	0.02	0.00
	322	0.58	0.31	0.21	0.06	0.04	0.00
	332	0.43	0.29	0.15	0.08	0.01	0.00
	333	0.50	0.23	0.11	0.03	0.04	0.00
	433	0.35	0.20	0.07	0.07	0.02	0.00
C	2211	0.63	0.38	0.19	0.10	0.04	0.00
	2221	0.44	0.26	0.18	0.06	0.05	0.00
	3321	0.34	0.19	0.05	0.04	0.02	0.00
D	21111	0.48	0.29	0.24	0.16	0.09	0.00
	22111	0.31	0.24	0.16	0.09	0.06	0.00
	22211	0.30	0.20	0.11	0.07	0.04	0.00
average		0.47	0.28	0.16	0.08	0.03	0.00

Table 5 shows the standard error for each class, where sample games have ordered branches. The result indicates branch ordering can significantly improve the accuracy (or efficiency) of the algorithm.

10 Conclusion

Since 1970s, combinatorial game theory has become the common fundamental mathematical model for the analysis of many intelligent games. Mean and temperature are the most important concept for hot combinatorial games.

In this paper, we present an efficient algorithm to calculate the mean and temperature of multi-branch hot games. Moreover, we implement the search algorithm in a game-independent search engine. The search engine has simple interface; computer game programs can apply the engine easily. The key feature of MT-engine is its ability to generate a path whose outcome is most likely to reduce the distance between the lower and upper walls.

MT-search can output high quality outcome by searching a portion of the game tree. Given a game G with T nodes, the algorithm can output an answer after visiting $4 \times \sqrt{T}$ nodes. The standard error of the answer is less than 5% of the range of the sample space temperature. Our experiment result also indicates the importance of branch ordering.

Acknowledgements

The author would like to thank the National Science Council of the Republic of China (Taiwan) for financial support of this research under contract number NSC 98-2221-E-346-004 and NSC 96-2221-E-346-004.

References

- [1] John Milnor, "Sums of Positional Games", in Kuhn and Tucker (eds.) "Contributions to the Theory of Games", Ann. Math. Studies #28, Princeton, 1953 pp 291-301.
- [2] Olof Hanner, "Mean Play for Sums of Positional Games", Pacific J. Math. 9, 1959, pp 81-99.
- [3] E. R. Berlekamp, J. H. Conway, R. K. Guy, "Winning Ways for your Mathematical Plays", Academic Press, New York, 1982, Chapter 6.
- [4] J. H. Conway, "On Numbers and Games", Academic Press, New York, 1976, Chapter 9.
- [5] K. Kao, "Mean and Temperature Search for Combinatorial Games", JCIS 1998 Proceedings, Vol I, 00 389-392.